

Projet « Mon Miam Miam »

Spécifications Techniques des API

Module Expérience Étudiant & Commandes

Etudiante	Mukendi Kanyiki Wivine
Rôle	Conception des API RESTful
Version	V1.2
Date	22 octobre 2026

Table des matières

1 Introduction et périmètre du module **3**
1.1 Périmètre couvert . . . 3
1.2 Hors périmètre . . . 3

2 Objectifs du projet et d'apprentissage **3**
2.1 Objectifs du système (Mon Miam Miam) . . . 3
2.2 Objectifs d'apprentissage technique (conception d'API) . . . 3

3 Normes techniques et conventions **4**
3.1 Format et échanges . . . 4
3.2 Authentification . . . 4
3.3 Format des réponses . . . 4
3.4 Pagination et filtres . . . 5
3.5 Codes de réponse HTTP . . . 5

4 Règles de gestion **5**
4.1 Points de fidélité . . . 5
4.2 Statuts d'une commande . . . 6
4.3 Statuts d'une réclamation . . . 6
4.4 Autres règles . . . 6

5 Sécurité et conformité RGPD **6**

6 Récapitulatif des endpoints **7**

7 Spécifications détaillées des endpoints **8**
7.1 Module Authentification & gestion de profil . . . 8
7.1.1 Inscription d'un étudiant . . . 8
7.1.2 Connexion de l'étudiant . . . 9
7.1.3 Déconnexion . . . 10
7.1.4 Demande de récupération du mot de passe . . . 10
7.1.5 Réinitialisation du mot de passe . . . 11
7.1.6 Lecture du profil . . . 11
7.1.7 Mise à jour du profil . . . 12
7.2 Module Carte & menu du jour . . . 13
7.2.1 Consultation du menu du jour . . . 13
7.3 Module Panier & passation de commandes . . . 14
7.3.1 Validation d'une commande . . . 14
7.3.2 Historique des commandes . . . 15
7.3.3 Détail d'une commande . . . 16
7.3.4 Commentaire après livraison . . . 17
7.4 Module Réclamations . . . 18
7.4.1 Dépôt d'une réclamation . . . 18
7.4.2 Liste de mes réclamations . . . 19
7.4.3 Suivi d'une réclamation . . . 19

8 Diagrammes de cas d'utilisation et de classes **21**
8.1 Diagramme de cas d'utilisation . . . 21
8.2 Diagramme de classes . . . 23

9 Diagrammes de séquence **25**
9.1 Inscription d'un étudiant 25
9.2 Connexion d'un étudiant 26
9.3 Validation d'une commande avec points de fidélité 27

10 Catalogue des codes d'erreur **28**

11 Intégration avec les autres modules **28**

12 Plan de tests des API **28**

1 Introduction et périmètre du module

Le présent document détaille la conception technique des interfaces de programmation (API RESTful) destinées à l'espace étudiant de l'application **Mon Miam Miam** pour le restaurant *ZeDuc@Space*.

1.1 Périmètre couvert

Ce module couvre l'intégralité du cycle d'interaction de l'étudiant :

- L'inscription, la connexion, la déconnexion et la récupération du mot de passe, avec validation du mot de passe (au moins une majuscule et un chiffre).
- La consultation et la mise à jour du profil (nom, téléphone, localisation de livraison).
- La consultation dynamique du menu du jour (plats et boissons).
- La création de commande à partir du panier (sur place avec heure d'arrivée, ou en livraison).
- L'application des points de fidélité lors de la commande (utilisation et gain de points).
- L'historique des commandes, le détail d'une commande et le commentaire laissé une fois la commande livrée.
- Le dépôt et le suivi des réclamations relatives aux commandes.

1.2 Hors périmètre

Les fonctionnalités suivantes sont traitées dans d'autres modules du projet, avec lesquels ce module est lié (voir la section 11) :

- **Module Fidélité, Parrainage & Animation :** consultation du solde et de l'historique des points, parrainage, promotions, événements et mini-jeux, classement du Top 10 des meilleurs clients, statistiques, et paiement par agrégateur (bonus) ;
- **Module Back-Office & Administration :** gestion des commandes et du menu par les employés, le gérant et l'administrateur, traitement des réclamations, comptes employés.

2 Objectifs du projet et d'apprentissage

2.1 Objectifs du système (Mon Miam Miam)

L'application vise à :

- améliorer le suivi et la fluidité des commandes ;
- optimiser la gestion et l'attribution des livraisons ;
- assurer une traçabilité complète des ventes et des encaissements ;
- offrir une vue en temps réel des statistiques d'activité ;
- permettre la création de promotions, d'événements et de mini-jeux pour la fidélisation.

2.2 Objectifs d'apprentissage technique (conception d'API)

Du point de vue de l'ingénierie logicielle, la conception de ce module permet de :

- maîtriser la spécification rigoureuse d'API RESTful (méthodes HTTP, codes de statut, formats JSON) ;
- modéliser les interactions entre les acteurs (étudiants, système) et les composants (frontend, backend, base de données) ;

— intégrer les contraintes de sécurité dès la phase de conception (gestion des accès, chiffrement, validation).

3 Normes techniques et conventions

3.1 Format et échanges

— **URL de base :** `https://api.zeduc-space.com/api` (exemple de production) ; en local : `http://localhost:8000/api`. Toutes les routes de ce document sont données relativement à ce domaine.

— **Format des données :** JSON encodé en UTF-8. En-têtes requis :

  — `Content-Type: application/json`

  — `Accept: application/json`

— **Nommage :** champs en `snake_case`, en français, cohérents avec la base de données.

— **Dates et heures :** dates au format ISO 8601 avec fuseau `Africa/Douala` (UTC+1), par exemple `2026-10-22T11:30:00+01:00` ; heures simples au format `HH:MM`.

— **Montants :** en francs CFA (FCFA), nombres entiers, sans décimale.

3.2 Authentification

— Jeton d'accès de type **Bearer Token**, généré par *Laravel Sanctum* (retenu pour son intégration native à Laravel et la révocation simple d'un jeton).

— En-tête à envoyer sur les routes protégées : `Authorization: Bearer <token>`.

— Durée de vie du jeton : 24 heures (paramétrable). La déconnexion révoque le jeton courant.

— Routes publiques : inscription, connexion, mot de passe oublié, réinitialisation, menu du jour. Toutes les autres routes exigent un jeton valide.

3.3 Format des réponses

Toutes les réponses suivent la même enveloppe. En cas de succès, les données utiles sont dans le champ `data` :

```
{
  "success": true,
  "message": "Opération réalisée avec succès.",
  "data": { }
}
```

En cas d'erreur, le champ `code` est un identifiant stable exploitable par le frontend (voir la section 10) et `errors` détaille les champs invalides (erreurs 422) :

```
{
  "success": false,
  "message": "Les données fournies sont invalides.",
  "code": "VALIDATION_ERROR",
  "errors": {
    "email": ["Cette adresse email est déjà utilisée."]
  }
}
```

3.4 Pagination et filtres

Les listes (historique des commandes, réclamations) sont paginées avec les paramètres `page` (défaut 1) et `per_page` (défaut 10, maximum 50). Elles renvoient un objet `meta` ; ce format permet de remplir dynamiquement les tableaux en AJAX, comme demandé dans le cahier des charges.

```
"meta": { "page": 1, "per_page": 10, "total": 37, "last_page": 4 }
```

3.5 Codes de réponse HTTP

Code	Signification	Utilisation
200	OK	Lecture ou mise à jour réussie.
201	Created	Ressource créée (compte, commande, réclamation, avis).
401	Unauthorized	Jeton absent, invalide ou expiré ; identifiants incorrects.
403	Forbidden	Compte désactivé, ou accès à une ressource non autorisée.
404	Not Found	Ressource inexistante (ou appartenant à un autre étudiant).
409	Conflict	Action incompatible avec l'état actuel de la ressource.
422	Unprocessable Entity	Données invalides ou règle de gestion non respectée.
429	Too Many Requests	Limite de requêtes dépassée (en-tête `Retry-After`).
500	Internal Server Error	Erreur serveur inattendue (aucun détail technique n'est exposé).

4 Règles de gestion

4.1 Points de fidélité

Ces règles sont partagées avec le module Fidélité (consultation du solde, parrainage) ; ce module les applique lors de la passation de commande.

— **Gain :** 1 point par tranche entière de 1 000 FCFA du montant net payé : points gagnés = $\lfloor \text{montant net}/1000 \rfloor$.

— **Utilisation :** 15 points = 1 000 FCFA de réduction. Les points s'utilisent par paliers de 15 ; la réduction ne peut pas dépasser le sous-total de la commande.

— **Expiration :** les points non utilisés expirent au bout de 12 mois (traitement automatique par une tâche planifiée).

— **Atomicité :** l'enregistrement de la commande, la déduction et l'attribution des points se font dans une seule transaction SQL ; en cas d'échec, rien n'est enregistré.

Exemple de calcul (commande #204)

Élément	Valeur
Sous-total ($2 \times 3\,500 + 1 \times 1\,000$)	8 000 FCFA
Points utilisés	15 points
Réduction obtenue (15 points = 1 000 FCFA)	1 000 FCFA
Montant net à payer	7 000 FCFA
Points gagnés ($\lfloor 7000/1000 \rfloor$)	7 points
Solde : ancien − utilisés + gagnés	$30 - 15 + 7 = 22$ points

4.2 Statuts d'une commande

`Statut`	Signification
`en_attente`	Commande enregistrée, pas encore prise en charge par le restaurant.
`en_preparation`	Un employé prépare la commande.
`prete`	Commande prête à être servie ou à partir en livraison.
`en_livraison`	Commande en cours de livraison (mode `livraison` uniquement).
`livree`	Commande servie ou livrée. Seul ce statut autorise le commentaire.
`annulee`	Commande annulée par le restaurant.

4.3 Statuts d'une réclamation

Selon le cahier des charges, un employé propose une réponse et le gérant la valide ou la rejette. L'étudiant ne voit la réponse qu'une fois validée.

`Statut`	Signification
`ouverte`	Réclamation déposée, en attente de traitement.
`en_traitement`	Un employé examine la réclamation.
`resolue`	Réponse validée par le gérant et visible par l'étudiant.
`rejetee`	Réclamation refusée, avec le motif communiqué à l'étudiant.

4.4 Autres règles

— **Panier :** il est géré côté client (frontend) et envoyé en une seule fois à `POST /api/orders`. Le serveur revérifie la disponibilité et recalcule tous les prix : le frontend n'envoie jamais de prix.

— **Mode de service :** `sur_place` exige `heure_arrivee` (heure future, comprise dans les horaires d'ouverture définis par l'administrateur) ; `livraison` exige `adresse_livraison`.

— **Commentaire après livraison :** un seul commentaire par commande, uniquement si la commande est `livree`.

— **Réclamations :** une réclamation ne peut porter que sur une commande de l'étudiant connecté, et une seule réclamation non clôturée est autorisée par commande.

5 Sécurité et conformité RGPD

— **Transport :** HTTPS obligatoire en production ; CORS limité au domaine du frontend.

— **Mots de passe :** hachés (bcrypt via `Hash::make` de Laravel), jamais stockés ni renvoyés en clair, jamais écrits dans les journaux.

— **Limitation de débit :** 5 tentatives de connexion par minute et par adresse IP ; 60 requêtes par minute pour les autres routes (erreur 429).

— **Validation :** toutes les entrées sont validées côté serveur (*Form Requests* Laravel) ; accès à la base par Eloquent/requêtes préparées contre l'injection SQL.

— **Contrôle d'accès :** un étudiant n'accède qu'à ses propres ressources. Les routes utilisent `/me` plutôt qu'un identifiant utilisateur, et une ressource d'un autre étudiant renvoie 404.

— **Anti-énumération :** `POST /api/auth/forgot-password` répond toujours de la même façon, que l'adresse existe ou non ; le message d'échec de connexion est générique.

— **Minimisation des données (RGPD) :** les réponses n'exposent que les données de l'étudiant connecté ; aucun email ni téléphone d'un autre utilisateur n'est divulgué. Le consentement aux cookies est recueilli côté frontend via la fenêtre prévue au cahier des charges.

6 Récapitulatif des endpoints

N°	Méthode	Route	Auth.	Description
1	POST	/api/auth/register	Non	Inscription
2	POST	/api/auth/login	Non	Connexion
3	POST	/api/auth/logout	Oui	Déconnexion
4	POST	/api/auth/forgot-password	Non	Demande de lien de récupération
5	POST	/api/auth/reset-password	Non	Nouveau mot de passe
6	GET	/api/auth/me	Oui	Lecture du profil
7	PATCH	/api/auth/me	Oui	Mise à jour du profil
8	GET	/api/menu/today	Non	Menu du jour
9	POST	/api/orders	Oui	Validation d'une commande
10	GET	/api/orders/history	Oui	Historique des commandes
11	GET	/api/orders/{id}	Oui	Détail d'une commande
12	POST	/api/orders/{id}/review	Oui	Commentaire après livraison
13	POST	/api/claims	Oui	Dépôt d'une réclamation
14	GET	/api/claims	Oui	Liste de mes réclamations
15	GET	/api/claims/{id}	Oui	Suivi d'une réclamation

Note d'implémentation Laravel : ordre des routes

Dans `routes/api.php`, la route `/orders/history` doit être déclarée *avant* `/orders/{id}`. Sinon Laravel interprète le mot « history » comme un identifiant de commande.

```
Route::middleware('auth:sanctum')->group(function () {
    Route::get('/orders/history', [OrderController::class, 'history']);
    Route::get('/orders/{id}',    [OrderController::class, 'show']);
    Route::post('/orders',        [OrderController::class, 'store']);
});
```

7 Spécifications détaillées des endpoints

7.1 Module Authentification & gestion de profil

7.1.1 Inscription d'un étudiant

Route	`POST /api/auth/register`
Authentification	Non
Description	Permet la création d'un compte étudiant. Le mot de passe doit comporter au moins une majuscule et au moins un chiffre.

Règles de validation :

Champ	Type	Requis	Règles
`nom`	chaîne	Oui	2 à 100 caractères
`email`	chaîne	Oui	Adresse email valide, unique
`mot_de_passe`	chaîne	Oui	8 caractères minimum, au moins une majuscule et un chiffre
`mot_de_passe_confirmation`	chaîne	Oui	Identique à `mot_de_passe`
`numero_telephone`	chaîne	Oui	Format international (ex. +237600000000)
`localisation`	chaîne	Oui	3 à 255 caractères ; adresse utilisée par défaut pour les livraisons

Corps de la requête :

```
{
  "nom": "Mukendi Kanyiki Wivine",
  "email": "wivine.kanyiki@ucac-icam.com",
  "mot_de_passe": "MiamMiam2026",
  "mot_de_passe_confirmation": "MiamMiam2026",
  "numero_telephone": "+237600000000",
  "localisation": "Résidence La Terrasse, Yansoki"
}
```

Réponse de succès (201 Created) :

```
{
  "success": true,
  "message": "Compte créé avec succès.",
  "data": {
    "user_id": 12,
    "nom": "Mukendi Kanyiki Wivine",
    "email": "wivine.kanyiki@ucac-icam.com",
    "numero_telephone": "+237600000000",
    "localisation": "Résidence La Terrasse, Yansoki",
    "role": "etudiant",
    "points_fidelite": 0,
    "token": "1|bearer_token_string_example",
    "token_type": "Bearer"
  }
}
```

Erreurs possibles :

HTTP	Condition	Code métier
422	Champ manquant ou invalide, mot de passe non conforme, email déjà utilisé.	`VALIDATION_ERROR`
429	Trop de requêtes.	`TOO_MANY_REQUESTS`

Exemple de réponse d'erreur (422) :

```
{
  "success": false,
  "message": "Les données fournies sont invalides.",
  "code": "VALIDATION_ERROR",
  "errors": {
    "email": ["Cette adresse email est déjà utilisée."],
    "mot_de_passe": ["Le mot de passe doit contenir au moins une majuscule et un chiffre."]
  }
}
```

7.1.2 Connexion de l'étudiant

Route	`POST /api/auth/login`
Authentification	Non
Description	Connexion sécurisée et génération du jeton d'accès. La réponse contient le rôle de l'utilisateur, ce qui permet au frontend de le rediriger vers le bon espace.

Règles de validation :

Champ	Type	Requis	Règles
`email`	chaîne	Oui	Adresse email valide
`mot_de_passe`	chaîne	Oui	Non vide

Corps de la requête :

```
{
  "email": "wivine.kanyiki@ucac-icam.com",
  "mot_de_passe": "MiamMiam2026"
}
```

Réponse de succès (200 OK) :

```
{
  "success": true,
  "message": "Connexion réussie.",
  "data": {
    "user_id": 12,
    "nom": "Mukendi Kanyiki Wivine",
    "email": "wivine.kanyiki@ucac-icam.com",
    "role": "etudiant",
    "points_fidelite": 22,
    "token": "2|bearer_token_string_example",
    "token_type": "Bearer",
    "expires_at": "2026-10-23T11:30:00+01:00"
  }
}
```

Erreurs possibles :

HTTP	Condition	Code métier
401	Email ou mot de passe incorrect (message volontairement générique).	`INVALID_CREDENTIALS`
403	Compte désactivé (statut `inactif`).	`ACCOUNT_DISABLED`
422	Champ manquant ou mal formé.	`VALIDATION_ERROR`
429	Plus de 5 tentatives par minute.	`TOO_MANY_REQUESTS`

7.1.3 Déconnexion

Route	`POST /api/auth/logout`
Authentification	Oui (`Authorization: Bearer <token>`)
Description	Révoque le jeton courant. Aucun corps de requête.

Réponse de succès (200 OK) :

```
{
  "success": true,
  "message": "Déconnexion réussie.",
  "data": null
}
```

Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`

7.1.4 Demande de récupération du mot de passe

Route	`POST /api/auth/forgot-password`
Authentification	Non
Description	Envoie par email un lien de réinitialisation valable 60 minutes. La réponse est identique que l'adresse existe ou non, afin d'empêcher la découverte des comptes existants.

Règles de validation :

Champ	Type	Requis	Règles
`email`	chaîne	Oui	Adresse email valide

Corps de la requête :

```
{ "email": "wivine.kanyiki@ucac-icam.com" }
```

Réponse de succès (200 OK) :

```
{
  "success": true,
  "message": "Si cette adresse existe, un lien de récupération vient d'être envoyé.",
  "data": null
}
```

Erreurs possibles :

HTTP	Condition	Code métier
422	Adresse email mal formée.	`VALIDATION_ERROR`
429	Trop de demandes.	`TOO_MANY_REQUESTS`

7.1.5 Réinitialisation du mot de passe

Route	`POST /api/auth/reset-password`
Authentification	Non
Description	Définit un nouveau mot de passe à partir du jeton reçu dans le lien envoyé par email. Tous les jetons d'accès existants du compte sont révoqués.

Règles de validation :

Champ	Type	Requis	Règles
`email`	chaîne	Oui	Adresse email du compte
`token`	chaîne	Oui	Jeton contenu dans le lien de l'email, valable 60 minutes
`mot_de_passe`	chaîne	Oui	Mêmes règles que l'inscription
`mot_de_passe_confirmation`	chaîne	Oui	Identique à `mot_de_passe`

Corps de la requête :

```
{
  "email": "wivine.kanyiki@ucac-icam.com",
  "token": "a1b2c3d4e5f6g7h8i9j0",
  "mot_de_passe": "NouveauMiam2026",
  "mot_de_passe_confirmation": "NouveauMiam2026"
}
```

Réponse de succès (200 OK) :

```
{
  "success": true,
  "message": "Mot de passe réinitialisé. Vous pouvez vous connecter.",
  "data": null
}
```

Erreurs possibles :

HTTP	Condition	Code métier
422	Mot de passe non conforme.	`VALIDATION_ERROR`
422	Jeton invalide ou expiré.	`INVALID_RESET_TOKEN`

7.1.6 Lecture du profil

Route	`GET /api/auth/me`
Authentification	Oui
Description	Retourne les informations du compte connecté.

Réponse de succès (200 OK) :

```
{
  "success": true,
  "data": {
    "user_id": 12,
    "nom": "Mukendi Kanyiki Wivine",
    "email": "wivine.kanyiki@ucac-icam.com",
    "numero_telephone": "+237600000000",
    "localisation": "Résidence La Terrasse, Yansoki",
    "role": "etudiant",
    "statut_compte": "actif",
    "points_fidelite": 22,
    "date_inscription": "2026-10-01T09:15:00+01:00"
  }
}
```


Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`

7.1.7 Mise à jour du profil

Route	`PATCH /api/auth/me`
Authentification	Oui
Description	Modifie partiellement le profil. Seuls les champs envoyés sont modifiés. L'email et le solde de points ne sont pas modifiables par cette route.

Règles de validation :

Champ	Type	Requis	Règles
`nom`	chaîne	Non	2 à 100 caractères
`numero_telephone`	chaîne	Non	Format international
`localisation`	chaîne	Non	3 à 255 caractères

Corps de la requête :


```
{ "localisation": "Campus UCAC-ICAM, Yansoki" }
```


Réponse de succès (200 OK) :


```
{
  "success": true,
  "message": "Profil mis à jour.",
  "data": {
    "user_id": 12,
    "nom": "Mukendi Kanyiki Wivine",
    "numero_telephone": "+237600000000",
    "localisation": "Campus UCAC-ICAM, Yansoki"
  }
}
```

Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
422	Valeur invalide.	`VALIDATION_ERROR`

7.2 Module Carte & menu du jour

7.2.1 Consultation du menu du jour

Route	`GET /api/menu/today`
Authentification	Non (route publique, utilisée par la page d'accueil)
Description	Récupère les plats et les boissons du jour. Les promotions et les événements sont fournis par le module Animation. Un plat marqué `disponible: false` est épuisé (mise à jour par les employés) : il reste affiché mais ne peut pas être commandé.

Réponse de succès (200 OK) :


```
{
  "success": true,
  "data": {
    "date": "2026-10-22",
    "menu": [
      {
        "id": 1,
        "categorie": "Plat Principal",
        "nom": "Poulet DG Miam Miam",
        "description": "Poulet, plantains mûrs et légumes sautés.",
        "prix": 3500,
        "disponible": true,
        "image_url": "https://zeduc-space.com/images/poulet_dg.png"
      },
      {
        "id": 2,
        "categorie": "Boisson",
        "nom": "Jus naturel Ananas-Gingembre",
        "description": "Pressé à la commande.",
        "prix": 1000,
        "disponible": true,
        "image_url": "https://zeduc-space.com/images/jus_ananas.png"
      }
    ]
  }
}
```


Erreurs possibles :

HTTP	Condition	Code métier
500	Erreur serveur inattendue.	`SERVER_ERROR`

7.3 Module Panier & passation de commandes

7.3.1 Validation d'une commande

Route	`POST /api/orders`
Authentification	Oui
Description	Valide le panier, gère le mode de service (sur place avec heure d'arrivée, ou livraison) et applique les points de fidélité. Le serveur recalcule les montants ; le frontend n'envoie aucun prix.

Règles de validation :

Champ	Type	Requis	Règles
`mode_service`	chaîne	Oui	`sur_place` ou `livraison`
`heure_arrivee`	chaîne `HH:MM`	Si `sur_place`	Heure future, dans les horaires d'ouverture
`adresse_livraison`	chaîne	Si `livraison`	5 à 255 caractères ; `null` si sur place
`commentaire`	chaîne	Non	255 caractères maximum (consigne de préparation)
`points_a_utiliser`	entier	Non	Multiple de 15, inférieur ou égal au solde et à la valeur du sous-total ; 0 par défaut
`articles`	tableau	Oui	De 1 à 30 lignes
`articles[].produit_id`	entier	Oui	Produit existant et disponible
`articles[].quantite`	entier	Oui	De 1 à 20

Corps de la requête :

```
{
  "mode_service": "sur_place",
  "heure_arrivee": "12:45",
  "adresse_livraison": null,
  "commentaire": "Servir le piment à part s'il vous plaît",
  "points_a_utiliser": 15,
  "articles": [
    { "produit_id": 1, "quantite": 2 },
    { "produit_id": 2, "quantite": 1 }
  ]
}
```

Réponse de succès (201 Created) :

```
{
  "success": true,
  "message": "Commande enregistrée avec succès.",
  "data": {
    "commande_id": 204,
    "statut": "en_attente",
    "mode_service": "sur_place",
    "heure_arrivee": "12:45",
    "adresse_livraison": null,
    "articles": [
      {
        "produit_id": 1,
        "nom": "Poulet DG Miam Miam",
        "quantite": 2,
        "prix_unitaire": 3500,
        "total_ligne": 7000
      },
```

```
      {
        "produit_id": 2,
        "nom": "Jus naturel Ananas-Gingembre",
        "quantite": 1,
        "prix_unitaire": 1000,
        "total_ligne": 1000
      }
    ],
    "sous_total": 8000,
    "points_utilises": 15,
    "reduction_points": 1000,
    "montant_net_a_payer": 7000,
    "statut_paiement": "en_attente",
    "points_gagnes": 7,
    "nouveau_solde_points": 22,
    "date_commande": "2026-10-22T11:30:00+01:00"
  }
}
```


Le champ `statut_paiement` est renseigné par le module de paiement (fonctionnalité bonus) ; sans ce module, il reste `en_attente` et le paiement se fait sur place.

Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
422	Champ manquant, mode de service incohérent avec les champs fournis.	`VALIDATION_ERROR`
422	Un article n'existe plus ou est épuisé.	`PRODUIT_INDISPONIBLE`
422	`points_a_utiliser` supérieur au solde.	`POINTS_INSUFFISANTS`
422	`points_a_utiliser` non multiple de 15, ou réduction supérieure au sous-total.	`POINTS_INVALIDES`
422	Heure d'arrivée hors des horaires d'ouverture.	`HORS_HORAIRES`

Exemple de réponse d'erreur métier (422) :


```
{
  "success": false,
  "message": "Solde de points insuffisant (disponible : 10, demandé : 15).",
  "code": "POINTS_INSUFFISANTS"
}
```


7.3.2 Historique des commandes

Route	`GET /api/orders/history`
Authentification	Oui
Description	Historique paginé des commandes de l'étudiant connecté, de la plus récente à la plus ancienne. Paramètres de requête : `page`, `per_page`, et `statut` (optionnel, une valeur de la liste des statuts).

Réponse de succès (200 OK) :


```
{
  "success": true,
  "data": [
    {
      "commande_id": 204,
```

```
      "date_commande": "2026-10-22T11:30:00+01:00",
      "statut": "livree",
      "mode_service": "sur_place",
      "nombre_articles": 3,
      "montant_net_a_payer": 7000,
      "points_gagnes": 7
    },
    {
      "commande_id": 187,
      "date_commande": "2026-10-19T12:10:00+01:00",
      "statut": "livree",
      "mode_service": "livraison",
      "nombre_articles": 2,
      "montant_net_a_payer": 4500,
      "points_gagnes": 4
    }
  ],
  "meta": { "page": 1, "per_page": 10, "total": 2, "last_page": 1 }
}
```


Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
422	`statut` inconnu, `per_page` supérieur à 50.	`VALIDATION_ERROR`

7.3.3 Détail d'une commande

Route	`GET /api/orders/{id}`
Authentification	Oui
Description	Retourne le détail complet d'une commande de l'étudiant connecté, avec son éventuel commentaire et la présence d'une réclamation. Une commande appartenant à un autre étudiant renvoie 404.

Réponse de succès (200 OK) :


```
{
  "success": true,
  "data": {
    "commande_id": 204,
    "statut": "livree",
    "mode_service": "sur_place",
    "heure_arrivee": "12:45",
    "adresse_livraison": null,
    "articles": [
      {
        "produit_id": 1,
        "nom": "Poulet DG Miam Miam",
        "quantite": 2,
        "prix_unitaire": 3500,
        "total_ligne": 7000
      },
      {
        "produit_id": 2,
        "nom": "Jus naturel Ananas-Gingembre",
        "quantite": 1,
        "prix_unitaire": 1000,
        "total_ligne": 1000
```

```
      }
    ],
    "sous_total": 8000,
    "points_utilises": 15,
    "reduction_points": 1000,
    "montant_net_a_payer": 7000,
    "points_gagnes": 7,
    "date_commande": "2026-10-22T11:30:00+01:00",
    "avis": null,
    "reclamation_id": null
  }
}
```


Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
404	Commande inexistante ou appartenant à un autre étudiant.	`NOT_FOUND`

7.3.4 Commentaire après livraison

Route	`POST /api/orders/{id}/review`
Authentification	Oui
Description	Permet à l'étudiant de laisser un commentaire (et une note facultative) une fois sa commande livrée, comme prévu au cahier des charges. Un seul commentaire par commande.

Règles de validation :

Champ	Type	Requis	Règles
`commentaire`	chaîne	Oui	3 à 500 caractères
`note`	entier	Non	De 1 à 5

Corps de la requête :


```
{
  "note": 4,
  "commentaire": "Très bon poulet, service rapide."
}
```


Réponse de succès (201 Created) :


```
{
  "success": true,
  "message": "Merci pour votre commentaire.",
  "data": {
    "commande_id": 204,
    "note": 4,
    "commentaire": "Très bon poulet, service rapide.",
    "date_avis": "2026-10-22T14:05:00+01:00"
  }
}
```

Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
404	Commande inexistante ou appartenant à un autre étudiant.	`NOT_FOUND`
409	La commande n'est pas encore livrée.	`COMMANDE_NON_LIVREE`
409	Un commentaire existe déjà pour cette commande.	`AVIS_DEJA_ENVOYE`
422	Commentaire vide ou note hors de 1 à 5.	`VALIDATION_ERROR`

7.4 Module Réclamations

7.4.1 Dépôt d'une réclamation

Route	`POST /api/claims`
Authentification	Oui
Description	Permet de signaler un problème lié à une commande de l'étudiant connecté. La réclamation est créée avec le statut `ouverte`.

Règles de validation :

Champ	Type	Requis	Règles
`commande_id`	entier	Oui	Commande existante appartenant à l'étudiant
`objet`	chaîne	Oui	3 à 100 caractères
`message`	chaîne	Oui	10 à 1000 caractères

Corps de la requête :

```
{
  "commande_id": 204,
  "objet": "Erreur sur la commande",
  "message": "Il manquait le jus d'ananas lors du service."
}
```

Réponse de succès (201 Created) :

```
{
  "success": true,
  "message": "Réclamation enregistrée.",
  "data": {
    "reclamation_id": 31,
    "commande_id": 204,
    "objet": "Erreur sur la commande",
    "message": "Il manquait le jus d'ananas lors du service.",
    "statut": "ouverte",
    "date_creation": "2026-10-22T13:20:00+01:00"
  }
}
```

Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
404	Commande inexistante ou appartenant à un autre étudiant.	`NOT_FOUND`
409	Une réclamation non clôturée existe déjà pour cette commande.	`RECLAMATION_DEJA_OUVERTE`
422	Champ manquant ou trop court.	`VALIDATION_ERROR`

7.4.2 Liste de mes réclamations

Route	`GET /api/claims`
Authentification	Oui
Description	Liste paginée des réclamations de l'étudiant connecté. Paramètres : `page`, `per_page`, et `statut` (optionnel : `ouverte`, `en_traitement`, `resolue`, `rejetee`).

Réponse de succès (200 OK) :

```
{
  "success": true,
  "data": [
    {
      "reclamation_id": 31,
      "commande_id": 204,
      "objet": "Erreur sur la commande",
      "statut": "en_traitement",
      "date_creation": "2026-10-22T13:20:00+01:00"
    }
  ],
  "meta": { "page": 1, "per_page": 10, "total": 1, "last_page": 1 }
}
```

Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
422	`statut` inconnu.	`VALIDATION_ERROR`

7.4.3 Suivi d'une réclamation

Route	`GET /api/claims/{id}`
Authentification	Oui
Description	Retourne le détail et l'état d'une réclamation. La réponse du restaurant n'apparaît que lorsqu'elle a été validée par le gérant (statut `resolue` ou `rejetee`) ; sinon `reponse` vaut `null`.

Réponse de succès (200 OK) :

```
{
  "success": true,
  "data": {
    "reclamation_id": 31,
```

```
    "commande_id": 204,
    "objet": "Erreur sur la commande",
    "message": "Il manquait le jus d'ananas lors du service.",
    "statut": "resolue",
    "date_creation": "2026-10-22T13:20:00+01:00",
    "reponse": {
      "message": "Nous nous excusons. Un jus offert vous attend à votre prochaine visite.",
      "date": "2026-10-22T16:40:00+01:00"
    }
  }
}
```


Erreurs possibles :

HTTP	Condition	Code métier
401	Jeton absent, invalide ou expiré.	`UNAUTHENTICATED`
404	Réclamation inexistante ou appartenant à un autre étudiant.	`NOT_FOUND`

8 Diagrammes de cas d'utilisation et de classes

Cette section présente la modélisation du module : les fonctionnalités offertes à l'étudiant (cas d'utilisation) et la structure des données qu'il manipule (classes).

8.1 Diagramme de cas d'utilisation

Le diagramme de la figure **??** montre ce que chaque acteur peut faire dans le module :

— le **visiteur** (non connecté) peut s'inscrire, se connecter, récupérer son mot de passe et consulter le menu du jour ;

— l'**étudiant** connecté hérite des droits du visiteur et peut en plus se déconnecter, gérer son profil, passer une commande, consulter son historique et le détail d'une commande, déposer et suivre une réclamation ;

— le **service d'envoi d'email** intervient lors de la récupération du mot de passe.

La commande *inclut* toujours le choix du mode de service (sur place ou livraison). L'utilisation de points de fidélité, le commentaire après livraison et le détail d'une commande sont des *extensions* facultatives.

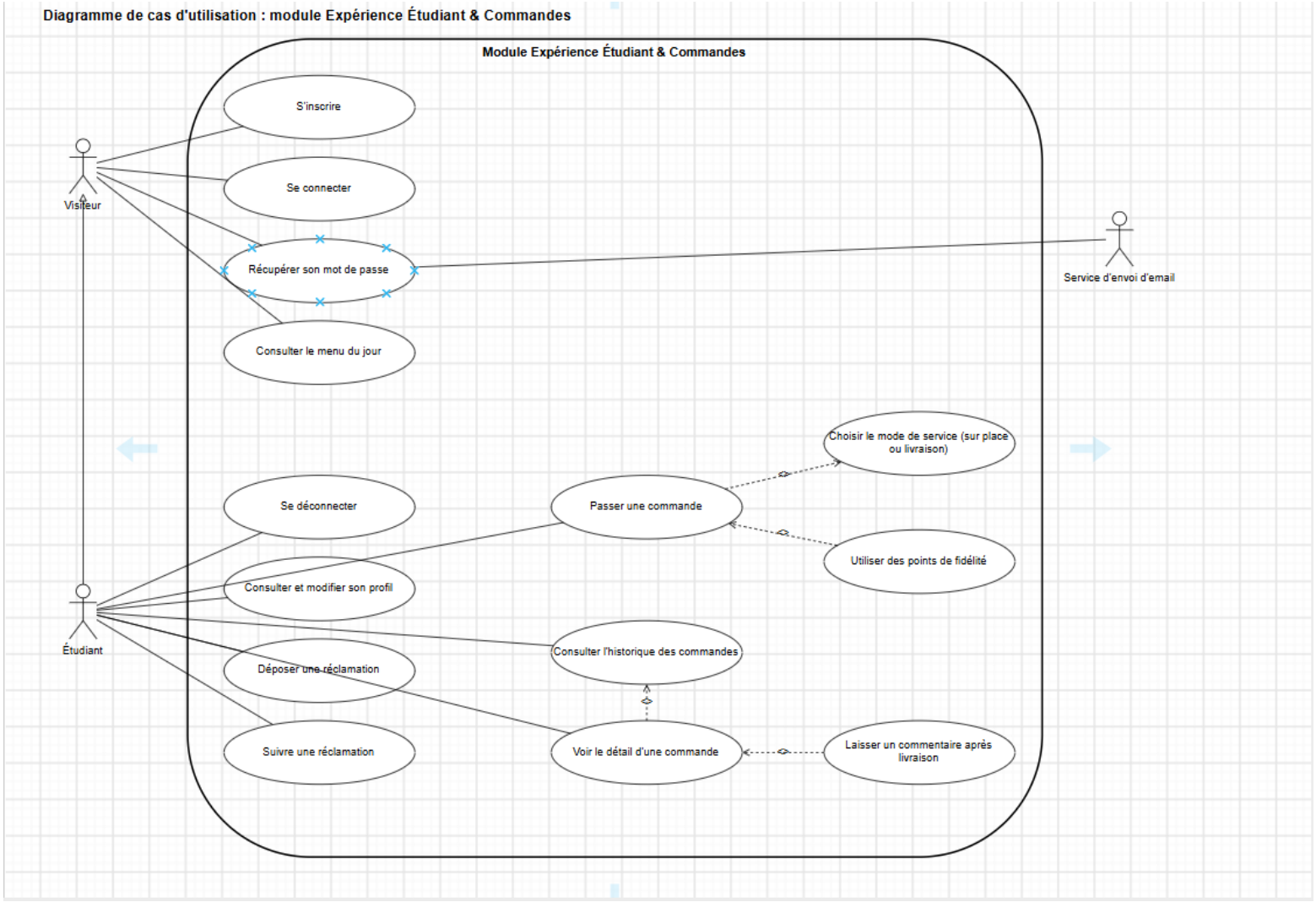


FIGURE 1 – Diagramme de classes du module Expérience Étudiant & Commandes

8.2 Diagramme de classes

Le diagramme de la figure 2 décrit les classes du module et leurs relations. Les noms d'attributs sont identiques aux champs des réponses des API (section 7).

- **Utilisateur** : le compte de l'étudiant, avec son rôle, son statut et son solde de points.
- **Produit** : un plat ou une boisson du menu, avec son prix en FCFA et sa disponibilité.
- **Commande** et **LigneCommande** : une commande contient une ou plusieurs lignes ; le prix unitaire est figé au moment de la commande.
- **MouvementPoints** : chaque gain, utilisation, expiration ou ajustement de points ; il peut être rattaché à une commande.
- **Reclamation** : le problème signalé sur une commande, avec la réponse du restaurant.
- **Avis** : le commentaire laissé une fois la commande livrée (au plus un par commande).

Les valeurs fixes (rôles, statuts, mode de service, type de mouvement) sont représentées par des énumérations, en bas du diagramme. Elles correspondent aux statuts de la section 4.

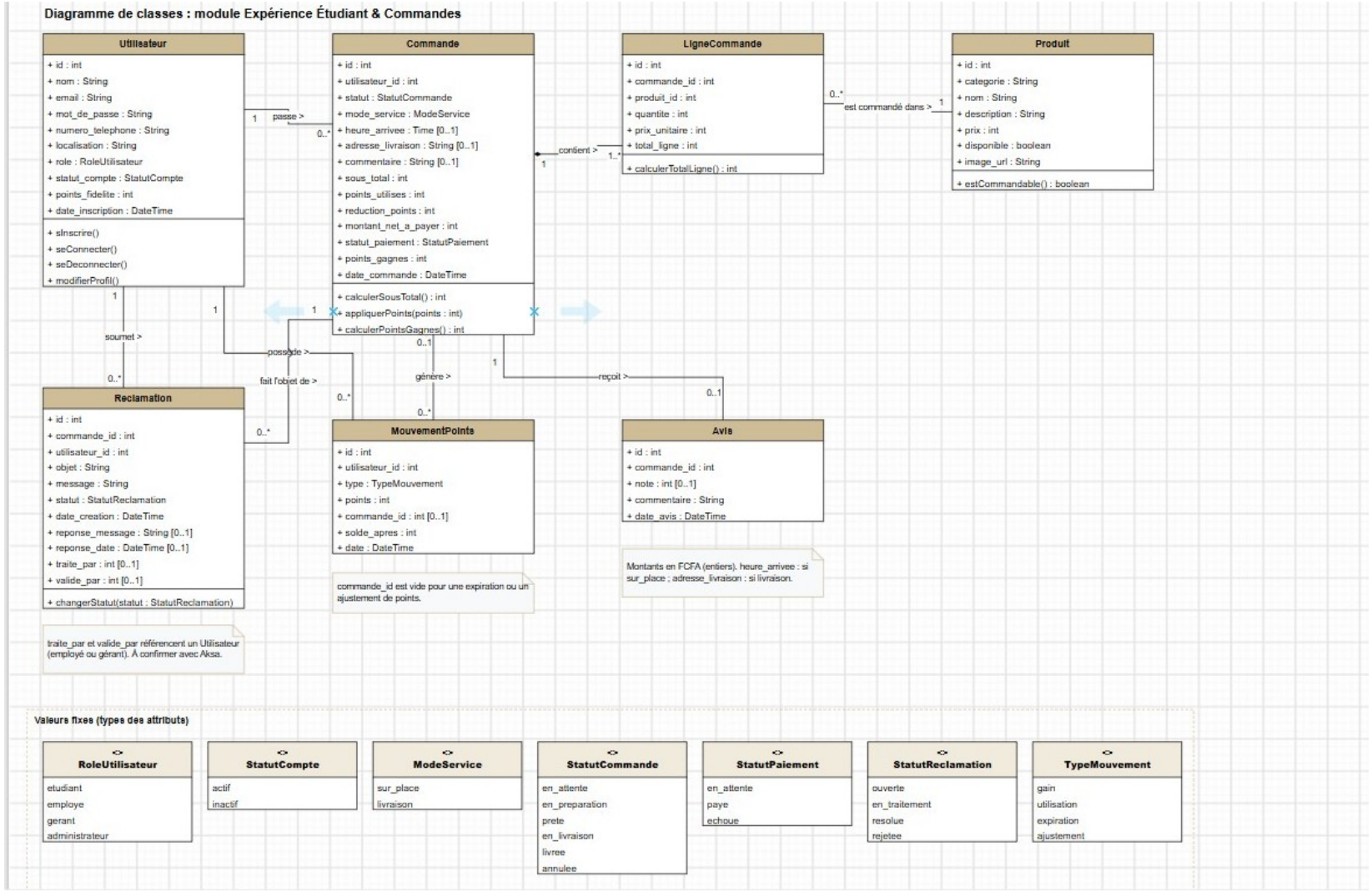


FIGURE 2 – Diagramme de classes du module Expérience Étudiant & Commandes

9 Diagrammes de séquence

Les trois diagrammes suivants, réalisés avec PlantUML, illustrent les échanges entre l'étudiant, le frontend, l'API Laravel et la base de données.

9.1 Inscription d'un étudiant

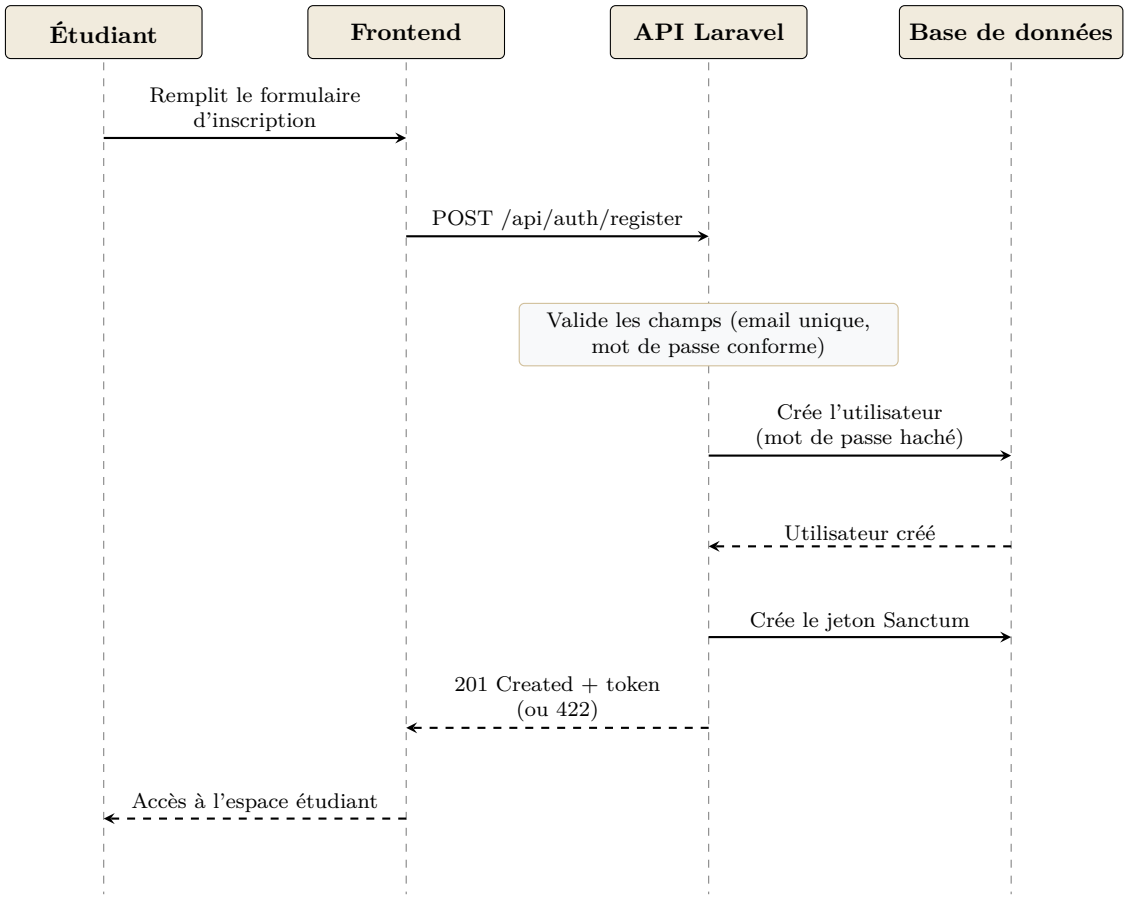


FIGURE 3 – Séquence d'inscription

9.2 Connexion d'un étudiant

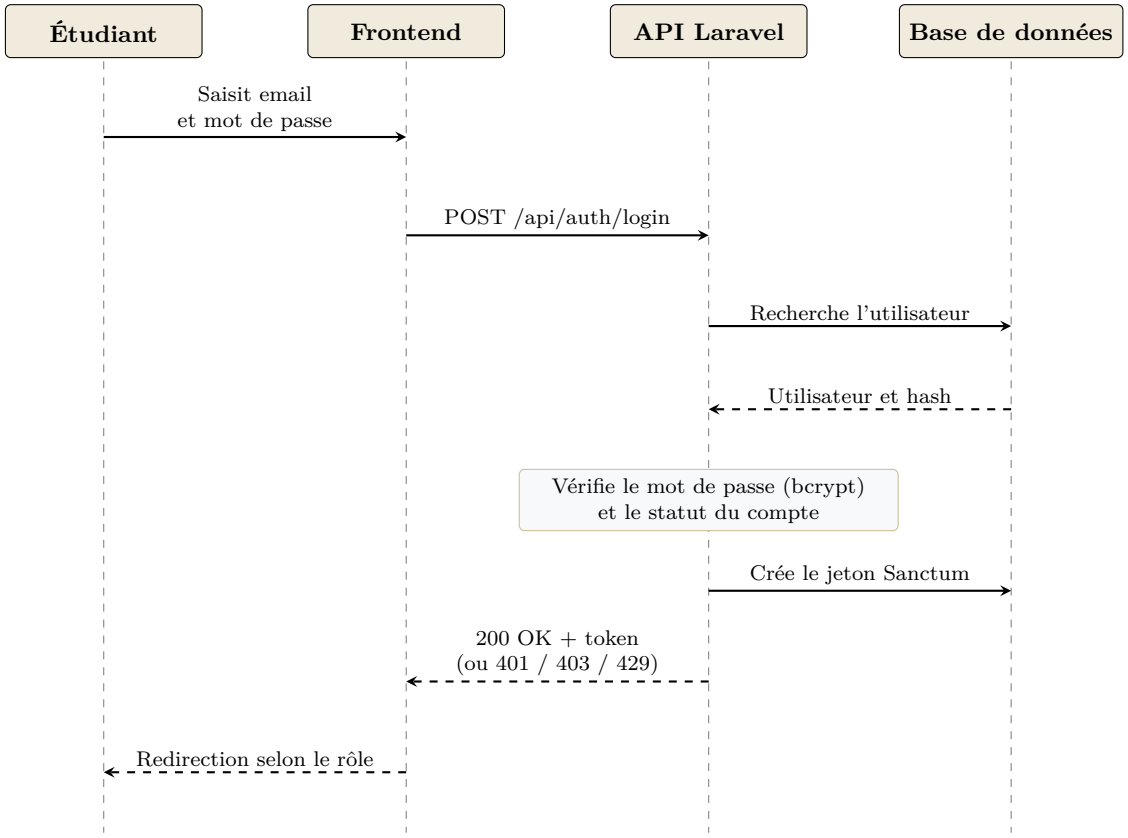


FIGURE 4 – Séquence de connexion

9.3 Validation d'une commande avec points de fidélité

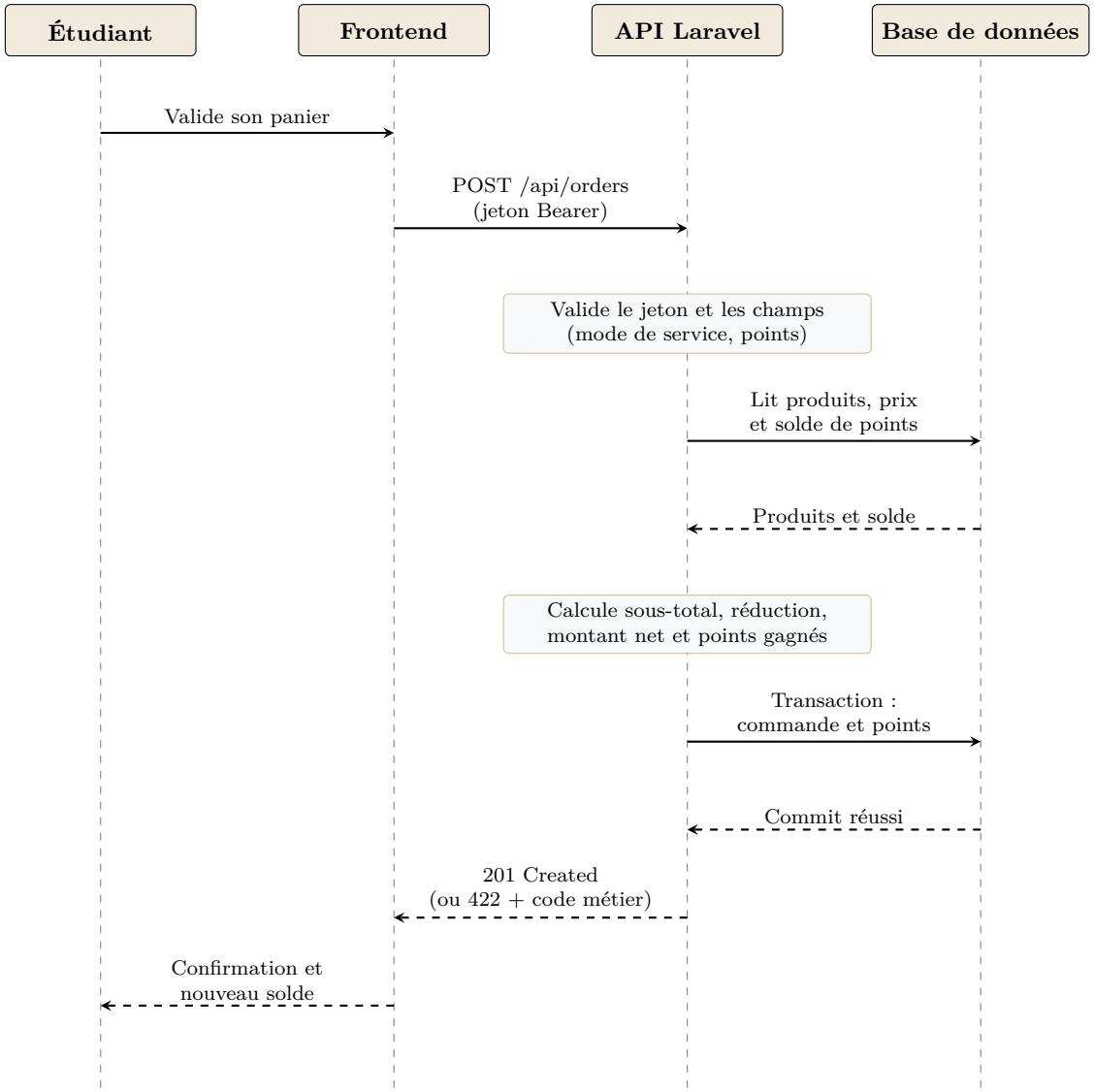


FIGURE 5 – Séquence de validation d'une commande (en cas d'erreur, la transaction est annulée et rien n'est enregistré)

10 Catalogue des codes d'erreur

Code	HTTP	Signification
VALIDATION_ERROR	422	Un ou plusieurs champs sont invalides (détail dans `errors`).
UNAUTHENTICATED	401	Jeton absent, invalide ou expiré.
INVALID_CREDENTIALS	401	Email ou mot de passe incorrect.
ACCOUNT_DISABLED	403	Compte désactivé par l'administration.
FORBIDDEN	403	Action non autorisée pour ce rôle.
NOT_FOUND	404	Ressource inexistante ou appartenant à un autre étudiant.
INVALID_RESET_TOKEN	422	Jeton de réinitialisation invalide ou expiré.
PRODUIT_INDISPONIBLE	422	Un article de la commande est épuisé ou supprimé.
POINTS_INSUFFISANTS	422	Solde de points inférieur aux points demandés.
POINTS_INVALIDES	422	Points non multiples de 15 ou réduction supérieure au sous-total.
HORS_HORAIRES	422	Heure d'arrivée hors des horaires d'ouverture.
COMMANDE_NON_LIVREE	409	Le commentaire exige une commande au statut `livree`.
AVIS_DEJA_ENVOYE	409	Un commentaire existe déjà pour cette commande.
RECLAMATION_DEJA_OUVERTE	409	Une réclamation non clôturée existe déjà pour cette commande.
TOO_MANY_REQUESTS	429	Limite de requêtes dépassée.
SERVER_ERROR	500	Erreur interne inattendue.

11 Intégration avec les autres modules

- **Module Fidélité, Parrainage & Animation :**
  - Les points sont consultés (solde, historique) par ce module ; ce module-ci les débite et les crédite lors de `POST /api/orders`, selon les règles de la section 4.
  - Après la première commande réussie d'un filleul, `POST /api/orders` déclenche la récompense de parrainage côté serveur.
  - À convenir avec ce module : l'ajout d'un champ optionnel `code_parrainage` à l'inscription.
  - Les promotions, événements, mini-jeux et le Top 10 sont fournis par ce module ; la page d'accueil du frontend les appelle directement.
- **Module Back-Office & Administration :** les employés et le gérant font évoluer le `statut` des commandes et des réclamations, en reprenant les statuts de la section 4 ; ils mettent à jour la disponibilité des plats, ce qui est répercuté sur `GET /api/menu/today`.
- **Paiement (bonus) :** le module d'agrégateur (ex. CinetPay) met à jour `statut_paiement` de la commande. Les points sont calculés sur le `montant_net_a_payer`.

12 Plan de tests des API

Les tests seront automatisés avec PHPUnit (tests de fonctionnalité Laravel) et rejoués manuellement avec une collection Postman partagée. Chaque cas ci-dessous correspond à un test.

N°	Scénario	Entrée / condition	Résultat attendu
1	Inscription valide	Données complètes, mot de passe `MiamMiam2026`	201, jeton retourné
2	Mot de passe sans majuscule ni chiffre	`miammiam`	422, `VALIDATION_ERROR`
3	Email déjà utilisé	Deuxième inscription avec le même email	422
4	Connexion valide	Bons identifiants	200, jeton retourné
5	Connexion échouée	Mauvais mot de passe	401, `INVALID_CREDENTIALS`
6	Limitation de débit	6 connexions en une minute	429
7	Route protégée sans jeton	`GET /api/orders/history`	401
8	Mot de passe oublié	Email inexistant	200, même message qu'un email existant
9	Commande sans points	2 plats + 1 boisson, `points_a_utiliser` à 0	201, `points_gagnes` = 8
10	Commande avec points	Solde 30, `points_a_utiliser` = 15	201, réduction 1 000, solde 22
11	Points insuffisants	Solde 10, `points_a_utiliser` = 15	422, `POINTS_INSUFFISANTS`
12	Points non multiples de 15	`points_a_utiliser` = 10	422, `POINTS_INVALIDES`
13	Plat épuisé	`produit_id` avec `disponible: false`	422, `PRODUIT_INDISPONIBLE`
14	Sur place sans heure	`mode_service` = `sur_place`, `heure_arrivee` absente	422
15	Commande d'un autre étudiant	`GET /api/orders/{id}` d'un autre compte	404
16	Commentaire avant livraison	Commande `en_attente`	409, `COMMANDE_NON_LIVREE`
17	Double commentaire	Deux envois sur la même commande	409 au second
18	Réclamation valide	Commande de l'étudiant	201, statut `ouverte`
19	Double réclamation	Deuxième réclamation sur la même commande ouverte	409